

SecureShare (SPS-System) Project Documentation

Project Overview

SecureShare is a secure, cloud-based file management and sharing system designed for organizations. It is built using **Node.js**, **Express**, and **EJS** for the frontend rendering, with **PostgreSQL (via Neon)** serving as the relational database and **Cloudinary** handling secure, distributed file storage.

Folder Structure

The project follows an MVC (Model-View-Controller) architecture:

```
SPS-System/
├── src/
│   ├── config/          # Configuration files (DB, Cloudinary, Environment variables)
│   ├── controllers/     # Request handlers (Auth, File, Folder, User, Org, Views)
│   ├── middleware/      # Express middlewares (Auth, Rate Limiting, File Upload)
│   ├── models/          # Database schemas and models
│   ├── public/          # Static assets (CSS, JS, Images)
│   ├── routes/          # API and View route definitions
│   ├── services/        # External integrations (Cloudinary, Email/Nodemailer)
│   ├── views/           # EJS templates for frontend rendering
│   ├── app.js           # Express application setup
│   └── server.js        # Server entry point
├── .env.example         # Example environment variables
└── package.json         # Dependencies and scripts
```

Technologies Used

- Node.js & Express:** Core backend framework for routing and API creation.
- EJS (Embedded JavaScript):** Templating engine used for server-side HTML rendering.
- PostgreSQL (Neon):** Relational database used for storing user data, file metadata, and audit logs.
- Cloudinary:** Cloud service used for storing the actual files securely.
- Nodemailer:** Used for sending transactional emails via SMTP.
- Bcrypt & JWT:** Used for password hashing and secure, stateless authentication.

Core Integrations

1. Environment Variables (.env and src/config/env.js)

The application uses the `dotenv` package to load variables from `.env`. The `src/config/env.js` file centralizes and validates all required environment variables at startup. If critical variables like `DATABASE_URL` or `CLOUDINARY_API_KEY` are missing, the server throws an error and prevents startup.

```
// src/config/env.js
require('dotenv').config();

const env = {
  NODE_ENV: process.env.NODE_ENV || 'development',
  PORT: parseInt(process.env.PORT, 10) || 3000,
  DATABASE_URL: process.env.DATABASE_URL,
  // ...
};
```

```
// Strict validation
if (!env.DATABASE_URL || env.DATABASE_URL.trim() === '') {
  throw new Error('Missing required env var: DATABASE_URL');
}
module.exports = env;
```

2. Neon Database Integration (src/config/db.js)

The database connection is established using the `pg` (node-postgres) package. It utilizes connection pooling to connect to a Neon serverless PostgreSQL instance using the `DATABASE_URL` .

```
// src/config/db.js
const { Pool } = require('pg');
const env = require('./env');

const pool = new Pool({
  connectionString: env.DATABASE_URL,
  ssl: { rejectUnauthorized: false }
});

pool.on('error', (err) => {
  console.error('Unexpected DB pool error', err);
});

module.exports = pool;
```

3. Cloudinary Integration (src/services/cloudinary.service.js)

Files uploaded by users are not stored on the local server. Instead, they are streamed as buffers directly to Cloudinary. The integration relies on `CLOUDINARY_CLOUD_NAME` , `CLOUDINARY_API_KEY` , and `CLOUDINARY_API_SECRET` .

```
// src/services/cloudinary.service.js
const cloudinary = require('../config/cloudinary');

async function uploadBuffer(buffer, options) {
  return new Promise((resolve, reject) => {
    const stream = cloudinary.uploader.upload_stream(options, (error, result) => {
      if (error) reject(error);
      else resolve(result);
    });
    stream.end(buffer);
  });
}
```

4. SMTP Email Verification (src/services/email.service.js)

Nodemailer is used to send organization invitations and password reset emails via SMTP. The transporter connects to the configured SMTP host and verifies the connection upon startup using `transporter.verify()` .

```
// src/services/email.service.js
const nodemailer = require('nodemailer');
const env = require('../config/env');

const transporter = nodemailer.createTransport({
```

```

    host: env.SMTP_HOST,
    port: env.SMTP_PORT,
    secure: env.SMTP_PORT === 465,
    auth: {
      user: env.SMTP_USER,
      pass: env.SMTP_PASS
    }
  });

// SMTP Verification
transporter.verify((error) => {
  if (error) {
    console.warn('Email transporter not ready:', error.message);
  } else {
    console.log('Email transporter ready');
  }
});

```

5. Granular File Permissions Hierarchy & Access Resolution (src/models/share.model.js)

To enforce secure file access across the organization, the system uses a custom permissions-hierarchy resolution method `checkAccess` . It evaluates user relationships relative to a file and returns a `resolved_permission` variable mapping to `'owner'` , `'admin'` , `'edit'` , `'download'` , or `'view'` . When user-specific and organization-wide sharing parameters overlap, the highest level of access is resolved (edit > download > view).

```

// src/models/share.model.js
async function checkAccess(file_id, user_id, org_id, role) {
  const result = await pool.query(`
    SELECT
      f.*,
      u.full_name as uploader_name,
      (SELECT permission FROM file_shares
       WHERE file_id = $1 AND share_type = 'org_wide' AND (expires_at IS NULL OR expires_at > now()))
    LIMIT 1) as org_wide_permission,
      (SELECT permission FROM file_shares
       WHERE file_id = $1 AND shared_with_user_id = $2 AND (expires_at IS NULL OR expires_at > now()))
    LIMIT 1) as user_permission
    FROM files f
    JOIN users u ON f.uploaded_by = u.id
    WHERE f.id = $1 AND f.org_id = $3 AND f.is_deleted = false
  `, [file_id, user_id, org_id]);

  const file = result.rows[0];
  if (!file) return null;

  // Resolve permission hierarchy
  let resolved = null;
  if (file.uploaded_by === user_id) {
    resolved = 'owner';
  } else if (role === 'admin' || role === 'super_admin') {
    resolved = 'admin';
  } else {
    const userPerm = file.user_permission;
    const orgPerm = file.org_wide_permission;

    if (userPerm || orgPerm) {

```

```

const levels = { 'view': 1, 'download': 2, 'edit': 3 };
const userLevel = userPerm ? levels[userPerm] : 0;
const orgLevel = orgPerm ? levels[orgPerm] : 0;

const maxLevel = Math.max(userLevel, orgLevel);
if (maxLevel === 3) resolved = 'edit';
else if (maxLevel === 2) resolved = 'download';
else if (maxLevel === 1) resolved = 'view';
}
}

if (!resolved) return null; // No access

file.resolved_permission = resolved;
return file;
}

```

6. Robust Cloudinary Download Redirect Streaming (src/controllers/file.controller.js)

When initiating secure downloads, the server generates a signed Cloudinary delivery URL. To ensure maximum reliability and avoid empty or broken payload issues from the storage CDN, the controller implements a recursive helper that follows up to 5 levels of HTTP/HTTPS redirects. Furthermore, it defers setting response headers (such as `Content-Disposition` and `Content-Type`) until a successful `200 OK` connection is established with the final redirect endpoint.

```

// src/controllers/file.controller.js (Excerpt)
function streamFromCloudinary(targetUrl, redirectCount = 0) {
  if (redirectCount > 5) {
    return res.status(500).json({ success: false, message: 'Too many redirects from storage' });
  }

  const protocol = targetUrl.startsWith('https') ? https : http;
  protocol.get(targetUrl, (cloudinaryRes) => {
    if (cloudinaryRes.statusCode >= 300 && cloudinaryRes.statusCode < 400 &&
cloudinaryRes.headers.location) {
      return streamFromCloudinary(cloudinaryRes.headers.location, redirectCount + 1);
    }

    if (cloudinaryRes.statusCode !== 200) {
      return res.status(500).json({ success: false, message: 'Failed to fetch file from storage'
});
    }

    // Set headers ONLY when we have a successful 200 OK stream from Cloudinary!
    const filename = file.original_name.replace(/"/g, '\\"');
    res.setHeader('Content-Disposition', `attachment; filename="${filename}"`);
    res.setHeader('Content-Type', file.mime_type || 'application/octet-stream');

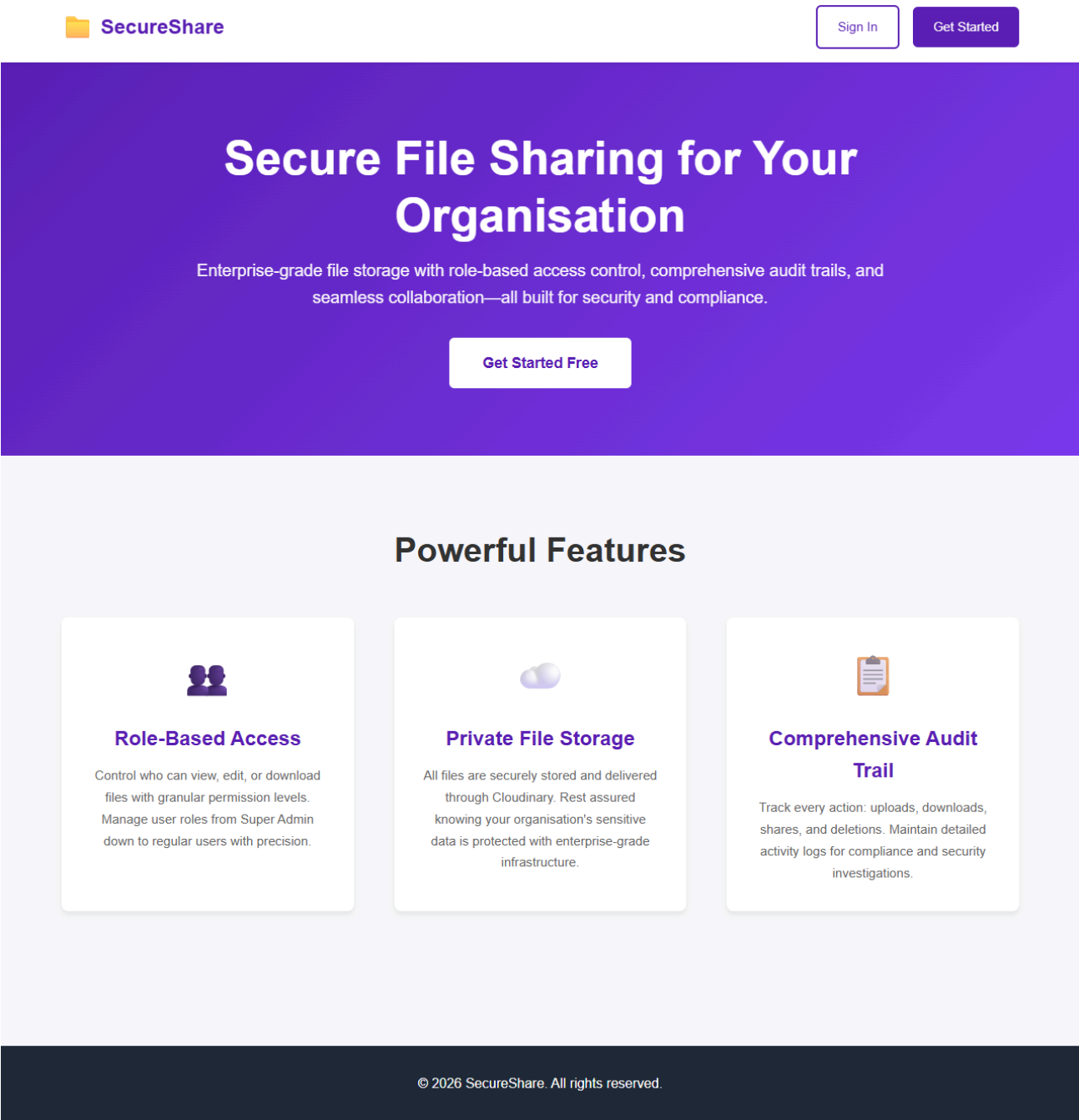
    cloudinaryRes.pipe(res);
  }).on('error', (err) => {
    console.error('Cloudinary download error:', err);
    res.status(500).json({ success: false, message: 'Error downloading file' });
  });
}

```

Web Pages and Code Mappings

1. Landing Page

- **Route:** GET /
- **Controller:** src/controllers/view.controller.js -> renderLanding
- **View Code:** src/views/landing.ejs



```
<!-- src/views/landing.ejs (Snippet) -->
<div class="hero">
  <h1>Secure File Sharing for Modern Teams</h1>
  <p>Store, share, and collaborate on files with enterprise-grade security.</p>
  <a href="/register" class="btn btn-primary">Get Started</a>
</div>
```

2. Login Page

- **Route:** GET /login
- **Controller:** src/controllers/view.controller.js -> renderLogin
- **View Code:** src/views/auth/login.ejs

The screenshot shows a web application titled "Secure File Sharing" in a blue header. The main content area is light gray and contains a white login card with a blue shadow. The card has the heading "Welcome Back" and the instruction "Sign in with your email and password". It features two input fields: "Email Address" with an eye icon and "Password" with a lock icon and a "Show" link. Below the inputs is a blue "Sign In" button. At the bottom of the card are two links: "Forgot Password?" and "Create Organisation".

```
<!-- src/views/auth/login.ejs (Snippet) -->
<form id="login-form">
  <div class="form-group">
    <i class="icon-email"></i>
    <input type="email" id="email" placeholder="Email Address" required>
  </div>
  <div class="form-group">
    <i class="icon-lock"></i>
    <input type="password" id="password" placeholder="Password" required>
  </div>
  <button type="submit" id="login-btn" class="btn-primary">Sign In</button>
</form>
```

3. Registration Page

- **Route:** GET /register
- **Controller:** src/controllers/view.controller.js -> renderRegister
- **View Code:** src/views/auth/register.ejs

Create Organisation

Organisation Name

Full Name

Email

Password

Confirm Password

Create Account

[Already have an account? Sign In](#)

```
<!-- src/views/auth/register.ejs (Snippet) -->
<form id="register-form">
  <div class="form-group">
    <label>Organisation Name</label>
    <input type="text" id="org_name" required>
  </div>
  <button type="submit" class="btn" style="width:100%;">Create Account</button>
</form>
```

4. User Dashboard

- **Route:** GET /dashboard
- **Controller:** src/controllers/view.controller.js -> renderDashboard
- **View Code:** src/views/dashboard.ejs

Welcome back, Test User!

Storage Used: 0.00 MB / 5120 MB

Recent Files

No recent files.

Upload File

Root Folders

No folders found.

```
<!-- src/views/dashboard.ejs (Snippet) -->
<div class="dashboard-header">
  <h2 class="text-2xl font-semibold mb-4">Welcome back, <%= user.full_name %></h2>
</div>
<div class="stats-grid">
  <div class="stat-card">
    <div class="stat-title">Total Files</div>
    <div class="stat-value"><%= stats.totalFiles %></div>
  </div>
</div>
```

5. File Detail Page

```
- **Route**: `GET /files/:id`
- **Controller**: `src/controllers/view.controller.js` -> `renderFileDetail`
- **View Code**: `src/views/files/detail.ejs`
- **Client JS Code**: `src/public/js/main.js`
```

This page renders full file metadata and dynamically displays interface sections based on the user's `resolvedPermission`:

1. **Conditional Sharing Card**: Rendered only when the user has `owner` or `admin` access:

```
`html
```

```
<!-- src/views/files/detail.ejs (Excerpt) -->
<% if (resolvedPermission === 'owner' || resolvedPermission === 'admin') { %>
  <div class="card">
    <h3>Share File</h3>
    <form id="share-form" data-file-id="<%= file.id %>">
      <!-- Specific User vs Org-Wide radio fields and email dropdown -->
    </form>
    <h4>Active Shares</h4>
```



```

        <!-- Active shares list with 'Revoke' button -->
    </div>
<% } %>

```

2. Inline Edit Details Form: Renders the toggleable tags and description editor for users with `'owner'`, `'admin'`, or `'edit'` permission:

```

<!-- src/views/files/detail.ejs (Excerpt) -->
<% if (resolvedPermission === 'owner' || resolvedPermission === 'admin' || resolvedPermission === 'edit')
{ %>
    <div id="edit-details-section" style="display: none; margin-top: 1.5rem; padding-top: 1.5rem; border-
top: 1px solid var(--border);">
        <h3>Edit File Details</h3>
        <form id="edit-file-form" data-file-id="<%= file.id %%">
            <input type="text" id="edit-tags" class="form-control" value="<%= Array.isArray(file.tags) ?
file.tags.join(', ') : '' %%">
            <textarea id="edit-desc" class="form-control" rows="3"><%= file.description || '' %>
        </textarea>
            <button type="submit" class="btn">Save Changes</button>
            <button type="button" id="cancel-edit-btn" class="btn btn-secondary">Cancel</button>
        </form>
    </div>
<% } %>

```

3. Frontend Interactions (`src/public/js/main.js`): Handles show, hide, and PATCH form submission to perform seamless inline metadata updates:

```

// src/public/js/main.js (Excerpt)
const editFileBtn = document.getElementById('edit-file-btn');
const editDetailsSection = document.getElementById('edit-details-section');
const cancelEditBtn = document.getElementById('cancel-edit-btn');
const editFileForm = document.getElementById('edit-file-form');

if (editFileBtn && editDetailsSection) {
    editFileBtn.addEventListener('click', () => {
        editDetailsSection.style.display = 'block';
        editFileBtn.style.display = 'none';
    });
}

if (editFileForm) {
    editFileForm.addEventListener('submit', async (e) => {
        e.preventDefault();
        const fileId = editFileForm.getAttribute('data-file-id');
        const tags = document.getElementById('edit-tags').value.split(',').map(t => t.trim()).filter(t =>
t !== '');
        const description = document.getElementById('edit-desc').value;

        try {
            await apiFetch(`/api/v1/files/${fileId}`, {
                method: 'PATCH',
                body: { tags, description }
            });
            showToast('File details updated successfully', 'success');
            setTimeout(() => window.location.reload(), 800);
        }
    });
}

```

```
    } catch (err) {}  
  });  
}
```

